



Olympos: Card Clash

JEU DE CARTES STRATÉGIQUE EN LIGNE

Étudiant : Enzo AIME

Formation : M2 Dev — Ynov Connect

Module : Développement d'application front / back

Document : Document de Cadrage

Date de rendu : 24 février 2026

DOCUMENT DE CADRAGE PROFESSIONNEL

Projet : Olympos: Card Clash

Rendu : 24/02/2026

Auteur : Enzo AIME

Dépôt GitHub : <https://github.com/Enzo91080/olympos>

1. Brief Projet

1.1 Présentation générale

Nom du projet : Olympos: Card Clash

Description : Olympos: Card Clash est un jeu de cartes stratégique en ligne, tour par tour, opposant deux joueurs (1v1) en temps réel. Les joueurs incarnent des dieux de la mythologie grecque et s'affrontent avec des decks composés de héros, créatures, sorts et artefacts divins. Chaque partie dure au maximum 30 minutes. Les decks sont limités à 30 cartes avec 2 exemplaires maximum par carte.

Problème résolu : Les jeux de cartes compétitifs existants (Hearthstone, Magic Arena) imposent des économies pay-to-win ou des interfaces surchargées. Olympos propose une expérience épurée, accessible et équitable, centrée sur la stratégie pure, avec une esthétique Dark Fantasy immersive inspirée de la mythologie grecque.

Public cible : Joueurs de 18 à 35 ans, fans de jeux de cartes stratégiques et de mythologie grecque, habitués aux interfaces gaming modernes.

1.2 Arborescence

```
Olympos: Card Clash |— /login ← Connexion / Inscription (public) |— /dashboard  
← Battlefront — Vue principale (protégé) | |— Stats du joueur (ELO, victoires,  
défaites) | |— Lancer une partie (matchmaking) | |— Accès rapide aux decks  
|— /deck-builder ← Armory — Liste des decks (protégé) | |— /deck-builder/:id ←  
Éditeur de deck |— /game/:id ← Game Board — Plateau de jeu (protégé) |— /  
leaderboard ← Pantheon — Classement ELO (protégé) |— /history ← Archives —  
Historique des parties (protégé)
```

1.3 Wireframes

Les wireframes ont été réalisés avec l'outil **Stitch** et exportés sous forme de fichiers HTML de référence dans le dossier `/design/`. Le système de design complet est documenté dans `design/aura_of_olympus/DESIGN.md`.

Page	Route	Fichier de référence
Login / Register	/login	design/login_register/code.html
Dashboard (Battlefront)	/dashboard	design/dashboard/code.html
Deck Builder (Armory)	/deck-builder	design/deck_builder/code.html
Game Board	/game/:id	design/game_board/code.html

1.4 Liste des fonctionnalités

Priorité	Fonctionnalité	Description	Rôle
● Must have	Inscription / Connexion	Créer un compte et se connecter via JWT	Tous
● Must have	Gestion de profil	Consulter et modifier son profil (username, avatar)	Joueur
● Must have	Galerie de cartes	Parcourir toutes les cartes disponibles avec filtres	Joueur
● Must have	Construction de deck	Créer et éditer un deck de 30 cartes (max 2 doublons)	Joueur
● Must have	Matchmaking ELO	File d'attente avec appariement (écart max 200 ELO)	Joueur
● Must have	Partie 1v1 temps réel	Jouer via WebSocket (jouer carte, passer tour, abandonner)	Joueur
● Must have	Fin de partie & ELO	Calcul du résultat et mise à jour du score ELO	Joueur
● Should have	Historique des parties	Consulter les parties passées avec résultats et actions	Joueur
● Should have	Classement (Leaderboard)	Afficher le top des joueurs par score ELO	Tous
● Should have	Oracle (IA)	Assistant IA analysant le deck et suggérant des améliorations	Joueur
● Should have	Reconnexion en partie	Rejoindre une partie après déconnexion (TTL Redis 10 min)	Joueur
● Nice to have	Boutique de cartes	Acheter des cartes avec une monnaie virtuelle	Joueur
● Nice to have	Animations de carte	Effets visuels lors du jeu d'une carte (Framer Motion)	Joueur
● Nice to have	Mode spectateur	Observer une partie en cours	Spectateur

2. Modélisation de la base de données

2.1 MCD — Modèle Conceptuel de Données

Association	Cardinalité
PLAYER — DECK	Un joueur possède 0 à N decks. Un deck appartient à 1 joueur.
PLAYER — GAME	Un joueur participe à 0 à N parties (en tant que player1 ou player2).
PLAYER — MATCHMAKING_QUEUE	Un joueur est présent dans 0 ou 1 file d'attente à la fois.
DECK — DECK_CARD — CARD	Un deck contient 1 à N cartes (via table de jonction, quantité 1 ou 2). Une carte peut être dans 0 à N decks.
GAME — GAME_ACTION	Une partie enregistre 0 à N actions. Une action appartient à 1 partie.

ATTRIBUTS DES ENTITÉS PRINCIPALES

Entité	Attributs
PLAYER	id, username, email, passwordHash, eloScore, avatarUrl, createdAt, updatedAt
CARD	id, name, cardType, manaCost, attack, defense, effectText, rarity, imageUrl, createdAt
DECK	id, playerId, name, isValid, createdAt, updatedAt
DECK_CARD	deckId, cardId, quantity (1-2)
GAME	id, player1Id, player2Id, deck1Id, deck2Id, winnerId, status, startedAt, endedAt
GAME_ACTION	id, gameId, playerId, cardId, actionType, turnNumber, payload (JSON), createdAt
MATCHMAKING_QUEUE	id, playerId, status, eloMin, eloMax, joinedAt

2.2 MLD — Modèle Logique de Données

```
player (id, username, email, passwordHash, eloScore, avatarUrl, createdAt, updatedAt)

card (id, name, cardType, manaCost, attack, defense, effectText, rarity, imageUrl, createdAt)

deck (id, #playerId, name, isValid, createdAt, updatedAt)

deck_card (#deckId, #cardId, quantity)

game (id, #player1Id, #player2Id, #deck1Id, #deck2Id, winnerId, status, startedAt, endedAt)

game_action (id, #gameId, #playerId, #cardId, actionType, turnNumber, payload, createdAt)

matchmaking_queue (id, #playerId, status, eloMin, eloMax, joinedAt)
```

Clés primaires soulignées — clés étrangères préfixées par #

2.3 MPD — Modèle Physique de Données

```

CREATE TABLE player (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username    VARCHAR(50)  NOT NULL UNIQUE,
  email       VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  elo_score   INTEGER      NOT NULL DEFAULT 1000,
  avatar_url  VARCHAR(500),
  created_at  TIMESTAMP    NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMP    NOT NULL DEFAULT NOW()
);

CREATE TABLE card (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(100) NOT NULL,
  card_type   VARCHAR(20)  NOT NULL
                    CHECK (card_type IN ('creature', 'spell', 'artifact')),
  mana_cost   INTEGER      NOT NULL,
  attack      INTEGER,
  defense     INTEGER,
  effect_text TEXT,
  rarity      VARCHAR(20)  NOT NULL
                    CHECK (rarity IN ('common', 'rare', 'epic', 'legendary')),
  image_url   VARCHAR(500),
  created_at  TIMESTAMP    NOT NULL DEFAULT NOW()
);

CREATE TABLE deck (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  player_id   UUID          NOT NULL REFERENCES player(id) ON DELETE CASCADE,
  name        VARCHAR(100) NOT NULL,
  is_valid    BOOLEAN       NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMP     NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMP     NOT NULL DEFAULT NOW()
);

CREATE TABLE deck_card (
  deck_id  UUID  NOT NULL REFERENCES deck(id) ON DELETE CASCADE,
  card_id  UUID  NOT NULL REFERENCES card(id) ON DELETE CASCADE,
  quantity INTEGER NOT NULL DEFAULT 1 CHECK (quantity BETWEEN 1 AND 2),
  PRIMARY KEY (deck_id, card_id)
);

CREATE TABLE game (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  player1_id  UUID          NOT NULL REFERENCES player(id),
  player2_id  UUID          NOT NULL REFERENCES player(id),
  deck1_id    UUID          NOT NULL REFERENCES deck(id),
  deck2_id    UUID          NOT NULL REFERENCES deck(id),
  winner_id   UUID          REFERENCES player(id),
  status      VARCHAR(20)  NOT NULL DEFAULT 'waiting'
                    CHECK (status IN ('waiting', 'in_progress', 'finished', 'abandoned')),
  started_at  TIMESTAMP,
  ended_at    TIMESTAMP

```



```

);

CREATE TABLE game_action (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  game_id     UUID          NOT NULL REFERENCES game(id) ON DELETE CASCADE,
  player_id   UUID          NOT NULL REFERENCES player(id),
  card_id     UUID          REFERENCES card(id),
  action_type VARCHAR(30) NOT NULL
              CHECK (action_type IN
                    ('play_card', 'attack', 'cast_spell', 'end_turn', 'surrender')),
  turn_number INTEGER      NOT NULL,
  payload     JSONB,
  created_at  TIMESTAMP    NOT NULL DEFAULT NOW()
);

CREATE TABLE matchmaking_queue (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  player_id   UUID          NOT NULL UNIQUE REFERENCES player(id) ON DELETE CASCADE,
  status      VARCHAR(20) NOT NULL DEFAULT 'waiting'
              CHECK (status IN ('waiting', 'matched', 'cancelled')),
  elo_min     INTEGER      NOT NULL,
  elo_max     INTEGER      NOT NULL,
  joined_at   TIMESTAMP    NOT NULL DEFAULT NOW()
);

```

3. Stack Technique

3.1 Frontend

Élément	Choix	Justification
Framework	React 18 + Vite	React est le standard de l'industrie pour les SPA complexes. Vite offre un HMR quasi-instantané, essentiel pour développer une interface gaming avec de nombreux composants interactifs.
Langage	TypeScript (strict)	Le typage strict prévient les erreurs de runtime sur les structures complexes (état de partie, cartes, actions) et améliore l'expérience développeur avec l'autocomplétion.
Styles	Tailwind CSS	Permet de styler en restant dans le JSX via des tokens (couleurs, border-radius), garantissant la cohérence du design system <i>Aura of Olympus</i> sur toute l'application.
Animations	Framer Motion	Bibliothèque d'animation déclarative React, indispensable pour les animations de cartes (retournement, déplacement) au cœur de l'expérience gaming.
État global	Zustand	Store minimaliste sans boilerplate (vs Redux). Adapté pour l'état du jeu en temps réel, des decks et de l'authentification.
WebSocket	Socket.io-client	Correspondance directe avec le backend Socket.io. Gère automatiquement la reconnexion et le fallback HTTP long-polling.
Routage	React Router v6	Standard de facto pour le routage React. Routes protégées (guard JWT) et routes dynamiques nativement supportées.
HTTP	Axios	Intercepteurs JWT centralisés sur toutes les requêtes et gestion des erreurs 401 (redirect login automatique).
Fonts	Noto Serif + Manrope	Noto Serif pour les titres (évocation des inscriptions antiques), Manrope pour le corps du texte (lisibilité moderne).

3.2 Backend

Élément	Choix	Justification
Runtime / Framework	Node.js + NestJS	Architecture modulaire (AuthModule, DeckModule, GameModule...) qui facilite la séparation des responsabilités. Guards, Pipes et Decorators idéaux pour la validation des actions de jeu côté serveur.
Langage	TypeScript	Cohérence full-stack avec le frontend. Les types Prisma générés sont partagés entre les services, réduisant les risques d'incohérence de données.
Authentification	JWT + bcrypt	JWT stateless (pas de session serveur). bcrypt pour le hachage des mots de passe avec sel. Guard NestJS protège toutes les routes décorées.
ORM	Prisma	Génère les types TypeScript depuis le schéma. Requêtes typées, autocomplétion, migrations versionnées.
Cache / Session	Redis 7	États de parties actives en Redis (TTL 10 min). Optimisé pour les lectures/écritures à haute fréquence lors des échanges d'actions.
WebSocket	Socket.io	Gestion des rooms par partie, broadcast ciblé aux deux joueurs uniquement. Synchronisation bidirectionnelle native.

3.3 Base de données

Élément	Choix	Justification
SGBD	PostgreSQL 15	Robustesse des contraintes relationnelles (intégrité des decks, unicité des files d'attente). Support natif du type JSONB pour le champ <code>payload</code> des GameActions.
Hébergement local	Docker Compose	Environnement reproductible avec <code>docker-compose.yml</code> orchestrant PostgreSQL 15 + Redis 7.
Hébergement prod	Railway	Déploiement cloud avec provisioning PostgreSQL natif et intégration GitHub Actions.

3.4 Outils & infrastructure

Élément	Choix
Versioning	Git + GitHub
Containerisation	Docker + Docker Compose (PostgreSQL 15 + Redis 7)
Déploiement Backend	Railway
Déploiement Frontend	Vercel
Gestion de projet	GitHub Projects

4. Fonctionnalité IA — L'Oracle

4.1 Description de la fonctionnalité

Fonctionnalité : L'Oracle est un assistant stratégique intégré au Deck Builder. Il analyse la composition du deck du joueur et génère des recommandations personnalisées : équilibre des types de cartes, courbe de mana, synergies manquantes, cartes à doubler ou retirer.

Problème résolu : La construction de deck est la phase la plus complexe pour les nouveaux joueurs. Sans guide, ils créent des decks sous-optimaux et perdent systématiquement, ce qui nuit à la rétention. L'Oracle démocratise l'accès à la stratégie avancée sans alourdir l'interface.

Moment d'intervention : Dans la page `/deck-builder`, un bouton "Consulter l'Oracle" apparaît une fois qu'au moins 10 cartes sont ajoutées au deck. L'Oracle s'affiche dans un panneau latéral sous forme de dialogue (style parchemin) avec ses recommandations.

4.2 Choix technique

Élément	Détail
Modèle IA	<code>gemma3:1b</code> via Ollama — exécuté 100 % en local, aucune clé API, aucun appel cloud
Intégration	Côté backend uniquement (NestJS — module <code>OracleModule</code>). Ollama est interrogé sur <code>http://localhost:11434</code> .
Endpoint	<code>POST /oracle/ask</code> — protégé par guard JWT
Protocole de réponse	Server-Sent Events (SSE) — les tokens arrivent en temps réel, token par token, jusqu'au signal <code>done: true</code>
Données en entrée	Question en texte libre du joueur (ex : conseils sur son deck, règles du jeu, stratégie)
Données en sortie	Flux de tokens streamé, rendu progressivement dans le composant Oracle du frontend

EXEMPLE D'INPUT ENVOYÉ AU MODÈLE

```
{ "question": "Mon deck manque de créatures, que me conseilles-tu ?" }
```

PROMPT SYSTÈME INJECTÉ CÔTÉ BACKEND

"Tu es l'Oracle de l'Olympe, un sage mystérieux qui guide les joueurs d'Olympos: Card Clash. Tu réponds en français, de façon concise et théâtrale. Tu maîtrises parfaitement les règles du jeu : 20 PV, 30 cartes par deck, mana croissant (max 10), summoning sickness, types créature / sort / artefact. Réponds toujours de façon utile et précise, en restant dans le thème de la mythologie grecque."

FLUX SSE — FORMAT DE RÉPONSE

```
data: {"token": "Mortel"}
data: {"token": ", écoute"}
data: {"token": " la sagesse de l'Olympe..."}
...
data: {"done": true}
```

4.3 Statut d'implémentation

La fonctionnalité Oracle est **réellement implémentée** : module NestJS avec streaming Ollama (`gemma3:1b`) côté backend, et composant Oracle avec rendu SSE temps réel intégré dans le Deck Builder côté frontend.